Level 2

Hackathon – Fundamentals of Web Development

Project: Personal Expense Tracker

Student Details

Student Name	R.Ayesha Siddiqua
Register Number	34625U50005
Department	II B.Sc. Artificial Intelligence
ID	626C56D13EA1E48A5EB4A30341DDE82A
College	M.M.E.S Women's Arts and Science College
Date of Submission	24/09/2026

Git-Hub Link:

https://github.com/<your-username>/personal-expense-tracker (replace with your repository link)

1. Problem Statement

Develop a personal expense tracking application that enables users to record income and expenses, organize transactions, monitor spending, and analyze their financial activity through summaries and charts. Managing money by memory or on paper is time-consuming and error-prone: it is hard to know, at a glance, how much has been earned, how much has been spent, where the money went, and what is left for the month.

An online expense tracker solves these problems. It lets a user record every transaction with its amount, date, category and description, search and filter past records instantly, and see totals and trends automatically through dynamic summaries and charts, without needing a server, a database, or an internet connection.

2. Objectives

- To design a clean, responsive and easy-to-use expense tracking website using HTML, CSS and JavaScript.
- To allow users to add, edit and delete income and expense transactions.
- To support category management, and keyword / type / category / date-range search and filtering.
- To dynamically calculate total income, total expenses, current balance and monthly summaries.
- To visualize spending through a category-wise chart and a month-wise income vs. expense chart.
- To persist all data in the browser using Local Storage, with no backend required.

Expected outcome: a fully working front-end web application that lets a user record, organize and analyze their personal finances end-to-end in the browser, demonstrating core web development concepts such as DOM manipulation, event handling, data persistence and chart-based data visualization.

3. Architecture

The application is a single-page web application that runs entirely in the browser. It is divided into three layers: the Presentation Layer (HTML and CSS), the Application Logic Layer (JavaScript modules) and the Data Layer (transaction and category data held in the browser's Local Storage).

Modules of the application

Module	Purpose	Main functions
Transaction Module	Add, edit and delete income/expense records with amount, date, category and description.	addTransaction(), editTx(), deleteTx()
Category Module	Maintain the list of expense/income categories used across the app.	populateCategorySelects(), renderCategoryList()
Filter / Search Module	Filter the transaction list by keyword, type, category and date range.	getFiltered(), renderTable()
Summary Module	Calculate totals, balance and month-wise financial summaries.	renderSummary(), renderMonthSummary()
Chart Module	Render the category-wise and month-wise charts using Chart.js.	renderCharts()

Technologies used and their roles

- **HTML5** – Defines the structure of the page: header, summary cards, transaction form, table, categories and charts.
- **CSS3** – Styles the interface: colours, cards, grid layout, buttons, hover effects and responsive behaviour.
- **JavaScript (ES6)** – Implements all logic: rendering transactions, filtering, calculations and charts.
- **Chart.js** – Renders the expense-by-category doughnut chart and the income-vs-expense bar chart.
- **Browser Local Storage** – Persists all transactions and categories between sessions, with no server.
- **VS Code** – Used for writing and testing the code.
- **Web browser** – Any modern browser (Chrome, Edge, Firefox) is used to execute the application.

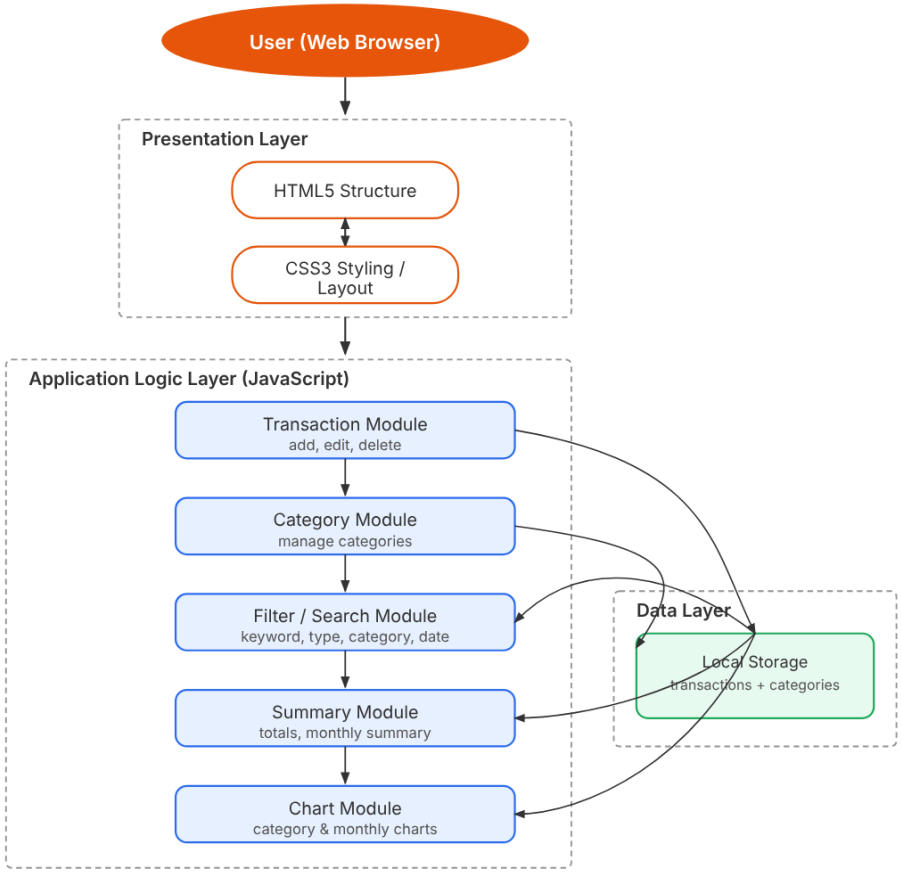


Figure 1: Application architecture and modules

User navigation and application flow

The user opens the dashboard and sees the summary cards, the transaction form and the transaction list. The user adds, edits, deletes, searches or filters transactions; every change is validated, saved to Local Storage, and immediately reflected in the totals, the monthly summary and the charts.

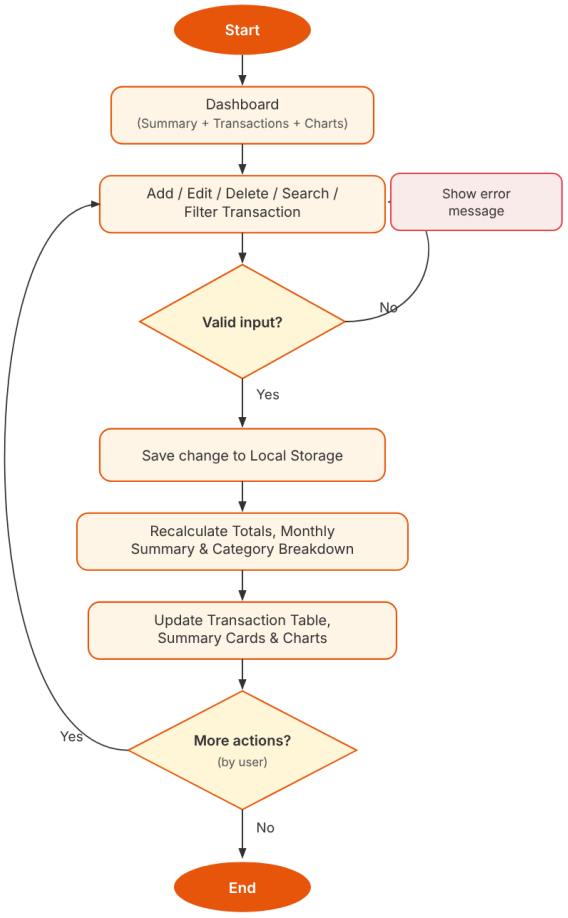


Figure 2: Application flow diagram (dashboard load to data update)

4. Practical Implementation

The application was developed step by step. First, the page layout was planned and the HTML skeleton was written with separate sections for the summary cards, the transaction form, the transaction table, category management and charts. Next, CSS was added to give the site a clean card-based layout with a blue colour theme and responsive grids. Finally, JavaScript was written to store transactions in Local Storage, render the table and charts dynamically, validate the transaction form, and recalculate every figure whenever data changes. Each feature was tested in the browser after it was built.

4.1 Features Implemented

Feature	Purpose
Add / edit / delete transactions	Users can record new income/expense entries and update or remove existing ones.
Income & expense recording	Each transaction stores amount, date, category, type (income/expense) and description.
Category management	Users can add new custom categories used while recording transactions.
Search & filter	Transactions can be filtered by keyword, type and category, together or separately.
Date range filter	Transactions can be filtered between a “from” and “to” date.
Dynamic totals	Total income, total expenses and current balance recalculate automatically.
Monthly summary	A card for each month shows income, expenses and balance for that month.
Local Storage persistence	All transactions and categories are saved in the browser and reload automatically.
Category & monthly charts	A doughnut chart (by category) and a bar chart (income vs. expense per month).
Responsive design	CSS Grid and Flexbox layout adjust to different screen sizes.

4.2 Source Code

The project is built as a single **index.html** file for simplicity. The sections below present its HTML structure, CSS styling and JavaScript logic separately for clarity.

HTML

HTML gives the page its structure: a header, three summary cards, the Add Transaction form, the Transactions panel with its filters and table, the Manage Categories panel, the Charts panel and the Monthly Summary panel.

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Personal Expense Tracker</title>
<script src="chart.umd.js"></script>
</head>
<body>
<header>
  <div>
```

```
    <h1>💰 Personal Expense Tracker</h1>
    <p>Record, organize and analyze your income &amp;amp; expenses</p>
  </div>
</header>

<div class="container">

  <div class="summary-grid">
    <div class="summary-card">
      <div class="label">Total Income</div>
      <div class="value income-value" id="totalIncome">₹0.00</div>
    </div>
    <div class="summary-card">
      <div class="label">Total Expenses</div>
      <div class="value expense-value" id="totalExpense">₹0.00</div>
    </div>
    <div class="summary-card">
      <div class="label">Current Balance</div>
      <div class="value balance-value" id="balance">₹0.00</div>
    </div>
  </div>

  <div class="panel">
    <h2>Add Transaction</h2>
    <form class="tx-form" id="txForm">
      <div class="field">
        <label>Description</label>
        <input type="text" id="desc" placeholder="e.g. Grocery shopping" required>
      </div>
      <div class="field">
        <label>Amount (₹)</label>
        <input type="number" id="amount" step="0.01" min="0.01" placeholder="0.00" required>
      </div>
      <div class="field">
        <label>Type</label>
        <select id="type">
          <option value="income">Income</option>
          <option value="expense" selected>Expense</option>
        </select>
      </div>
      <div class="field">
        <label>Category</label>
        <select id="category"></select>
      </div>
      <div class="field">
        <label>Date</label>
        <input type="date" id="date" required>
      </div>
      <button type="submit" class="btn-primary">Add</button>
    </form>
  </div>

  <div class="panel">
    <h2>Transactions</h2>
    <div class="filters">
      <input type="text" id="searchBox" placeholder="Search description...">
      <select id="filterType">
        <option value="">All Types</option>
        <option value="income">Income</option>
        <option value="expense">Expense</option>
      </select>
      <select id="filterCategory">
        <option value="">All Categories</option>
      </select>
      <input type="date" id="filterFrom" title="From date">
      <input type="date" id="filterTo" title="To date">
      <button id="clearFilters" style="background:#eef1f7;color:#44506b;">Clear Filters</button>
    </div>
    <table>
      <thead>
        <tr><th>Date</th><th>Description</th><th>Category</th><th>Type</th><th style="text-align:right;">Amount</th><th>Actions</th></tr>
      </thead>
      <tbody id="txTableBody"></tbody>
    </table>
    <div class="empty" id="emptyMsg" style="display:none;">No transactions found.</div>
  </div>

  <div class="panel">
    <h2>Manage Categories</h2>
    <form id="catForm" style="display:flex; gap:10px;">
      <input type="text" id="newCategory" placeholder="New category name" style="flex:1;">
      <button class="btn-primary" type="submit">Add Category</button>
    </form>
    <div id="categoryList" style="margin-top:12px; display:flex; gap:8px; flex-wrap:wrap;"></div>
```

```
  </div>

  <div class="panel">
    <h2>Charts</h2>
    <div class="charts-grid">
      <div>
        <h3 style="font-size:13px;color:var(--muted);">Expenses by Category</h3>
        <canvas id="categoryChart" height="220"></canvas>
      </div>
      <div>
        <h3 style="font-size:13px;color:var(--muted);">Income vs Expense by Month</h3>
        <canvas id="monthChart" height="220"></canvas>
      </div>
    </div>
  </div>

  <div class="panel">
    <h2>Monthly Summary</h2>
    <div class="month-summary-grid" id="monthSummary"></div>
  </div>

</div>

<footer>Personal Expense Tracker &copy; 2026 — Data stored locally in your browser</footer>

</body>
</html>
```

CSS

CSS defines the colour theme using CSS variables, lays out the summary cards and panels, styles the form, table, tags and buttons, and arranges the two charts side by side using CSS Grid.

```
:root{
    --bg:#f4f6fb; --card:#ffffff; --primary:#2f6fed; --primary-dark:#1f4fc4;
    --income:#1fab5e; --expense:#e5484d; --text:#1b2433; --muted:#6b7686;
    --border:#e3e7f0; --radius:14px;
  }
  *{box-sizing:border-box;}
  body{
    margin:0; font-family:'Segoe UI',Roboto,Arial,sans-serif; background:var(--bg); color:var(--text);
  }
  header{
    background:linear-gradient(135deg,var(--primary),var(--primary-dark)); color:#fff;
    padding:22px 28px; display:flex; align-items:center; justify-content:space-between;
  }
  header h1{margin:0; font-size:22px;}
  header p{margin:2px 0 0; font-size:13px; opacity:.85;}
  .container{max-width:1100px; margin:0 auto; padding:24px;}
  .summary-grid{display:grid; grid-template-columns:repeat(3,1fr); gap:16px; margin-bottom:22px;}
  .summary-card{background:var(--card); border-radius:var(--radius); padding:18px 20px; box-shadow:0 2px 10px rgba(20,30,60,.06); border:1px
solid var(--border);}
  .summary-card .label{font-size:12px; color:var(--muted); text-transform:uppercase; letter-spacing:.04em;}
  .summary-card .value{font-size:26px; font-weight:700; margin-top:6px;}
  .income-value{color:var(--income);}
  .expense-value{color:var(--expense);}
  .balance-value{color:var(--primary);}
  .panel{background:var(--card); border-radius:var(--radius); padding:20px 22px; box-shadow:0 2px 10px rgba(20,30,60,.06); border:1px solid
var(--border); margin-bottom:22px;}
  .panel h2{margin:0 0 14px; font-size:16px;}
  form.tx-form{display:grid; grid-template-columns:1fr 1fr 1fr 1fr auto; gap:10px; align-items:end;}
  .field{display:flex; flex-direction:column; gap:4px;}
  .field label{font-size:11px; color:var(--muted); text-transform:uppercase;}
  input,select{padding:9px 10px; border:1px solid var(--border); border-radius:8px; font-size:14px; background:#fbfcfe;}
  button{cursor:pointer; border:none; border-radius:8px; padding:10px 16px; font-size:14px; font-weight:600;}
  .btn-primary{background:var(--primary); color:#fff;}
  .btn-primary:hover{background:var(--primary-dark);}
  .filters{display:flex; gap:10px; flex-wrap:wrap; margin-bottom:14px;}
  .filters input,.filters select{min-width:150px;}
  table{width:100%; border-collapse:collapse; font-size:13.5px;}
  th,td{padding:10px 8px; text-align:left; border-bottom:1px solid var(--border);}
  th{color:var(--muted); font-weight:600; font-size:11.5px; text-transform:uppercase;}
  tr:hover{background:#f8faff;}
  .tag{padding:3px 9px; border-radius:20px; font-size:11.5px; font-weight:600;}
  .tag-income{background:#e4f8ee; color:var(--income);}
  .tag-expense{background:#fdeceb; color:var(--expense);}
  .amt-income{color:var(--income); font-weight:700;}
  .amt-expense{color:var(--expense); font-weight:700;}
  .row-actions button{padding:5px 10px; font-size:12px; margin-right:6px;}
  .btn-edit{background:#fff3da; color:#9a6a00;}
  .btn-del{background:#fde3e3; color:#b42318;}
  .charts-grid{display:grid; grid-template-columns:1fr 1fr; gap:18px;}
  .empty{color:var(--muted); text-align:center; padding:18px; font-size:13px;}
  .month-summary-grid{display:grid; grid-template-columns:repeat(auto-fill,minmax(170px,1fr)); gap:10px;}
  .month-card{border:1px solid var(--border); border-radius:10px; padding:10px 12px;}
  .month-card .m{font-size:12px; color:var(--muted);}
  .month-card .b{font-weight:700; font-size:15px; margin-top:4px;}
  footer{text-align:center; color:var(--muted); font-size:12px; padding:20px;}
```

JavaScript

JavaScript implements the application logic. Transactions and categories are stored in arrays and persisted to Local Storage on every change. Functions render the table, summary cards, monthly summary and the two Chart.js charts; a single set of filter listeners updates the table live as the user types or selects filters.

```
const STORAGE_KEY = 'pet_transactions';
const CATEGORY_KEY = 'pet_categories';
let transactions = JSON.parse(localStorage.getItem(STORAGE_KEY) || '[]');
let categories = JSON.parse(localStorage.getItem(CATEGORY_KEY) || 'null') ||
['Food','Transport','Salary','Rent','Shopping','Utilities','Entertainment','Health','Other'];
let editingId = null;
let categoryChart, monthChart;

function save(){
  localStorage.setItem(STORAGE_KEY, JSON.stringify(transactions));
  localStorage.setItem(CATEGORY_KEY, JSON.stringify(categories));
}

function uid(){ return 'tx_' + Date.now() + '_' + Math.floor(Math.random()*10000); }
function fmt(n){ return '₹' + Number(n).toFixed(2); }

function populateCategorySelects(){
  const selects = [document.getElementById('category'), document.getElementById('filterCategory')];
  selects.forEach((sel, idx) => {
    const keepFirst = idx === 1;
    sel.innerHTML = keepFirst ? '<option value="">All Categories</option>' : '';
    categories.forEach(c => {
      const o = document.createElement('option');
      o.value = c; o.textContent = c;
      sel.appendChild(o);
    });
  });
}

function renderCategoryList(){
  const wrap = document.getElementById('categoryList');
  wrap.innerHTML = '';
  categories.forEach(c => {
    const span = document.createElement('span');
    span.className = 'tag tag-expense';
    span.style.background = '#eef1f7';
    span.style.color = '#44506b';
    span.textContent = c;
    wrap.appendChild(span);
  });
}

function getFiltered(){
  const search = document.getElementById('searchBox').value.toLowerCase();
  const type = document.getElementById('filterType').value;
  const cat = document.getElementById('filterCategory').value;
  const from = document.getElementById('filterFrom').value;
  const to = document.getElementById('filterTo').value;
  return transactions.filter(t => {
    if (search && !t.description.toLowerCase().includes(search)) return false;
    if (type && t.type !== type) return false;
    if (cat && t.category !== cat) return false;
    if (from && t.date < from) return false;
    if (to && t.date > to) return false;
    return true;
  }).sort((a,b) => b.date.localeCompare(a.date));
}

function renderTable(){
  const tbody = document.getElementById('txTableBody');
  const list = getFiltered();
  tbody.innerHTML = '';
  document.getElementById('emptyMsg').style.display = list.length ? 'none' : 'block';
  list.forEach(t => {
    const tr = document.createElement('tr');
    tr.innerHTML = `
      <td>${t.date}</td>
      <td>${t.description}</td>
      <td>${t.category}</td>
      <td><span class="tag ${t.type === 'income' ? 'tag-income' : 'tag-expense'}">${t.type}</span></td>
      <td style="text-align:right;" class="${t.type === 'income' ? 'amt-income' : 'amt-expense'}">${t.type === 'income' ? '+' : '-
'}${fmt(t.amount)}</td>
      <td class="row-actions">
        <button class="btn-edit" onclick="editTx('${t.id}')">Edit</button>
```

```
        <button class="btn-del" onclick="deleteTx('${t.id}')">Delete</button>
      </td>`;
    tbody.appendChild(tr);
  });
}

function renderSummary(){
  const income = transactions.filter(t=>t.type==='income').reduce((s,t)=>s+Number(t.amount),0);
  const expense = transactions.filter(t=>t.type==='expense').reduce((s,t)=>s+Number(t.amount),0);
  document.getElementById('totalIncome').textContent = fmt(income);
  document.getElementById('totalExpense').textContent = fmt(expense);
  document.getElementById('balance').textContent = fmt(income - expense);
}

function renderMonthSummary(){
  const map = {};
  transactions.forEach(t => {
    const m = t.date.slice(0,7);
    if (!map[m]) map[m] = {income:0, expense:0};
    map[m][t.type] += Number(t.amount);
  });
  const wrap = document.getElementById('monthSummary');
  wrap.innerHTML = '';
  Object.keys(map).sort().reverse().forEach(m => {
    const bal = map[m].income - map[m].expense;
    const div = document.createElement('div');
    div.className = 'month-card';
    div.innerHTML = `<div class="m">${m}</div><div class="b" style="color:${bal>=0?'#1fab5e':'#e5484d'}">${fmt(bal)}</div>
      <div style="font-size:11px;color:#6b7686;">In: ${fmt(map[m].income)} / Out: ${fmt(map[m].expense)}</div>`;
    wrap.appendChild(div);
  });
}

function renderCharts(){
  const catTotals = {};
  transactions.filter(t=>t.type==='expense').forEach(t => {
    catTotals[t.category] = (catTotals[t.category]||0) + Number(t.amount);
  });
  const catLabels = Object.keys(catTotals);
  const catData = Object.values(catTotals);
  const palette = ['#2f6fed','#e5484d','#1fab5e','#f5a623','#9b5de5','#00bcd4','#ff6f91','#6b7686'];

  if (categoryChart) categoryChart.destroy();
  categoryChart = new Chart(document.getElementById('categoryChart'), {
    type: 'doughnut',
    data: { labels: catLabels, datasets: [{ data: catData, backgroundColor: palette }] },
    options: { plugins: { legend: { position: 'bottom', labels:{boxWidth:12,font:{size:11}} } } }
  });

  const monthMap = {};
  transactions.forEach(t => {
    const m = t.date.slice(0,7);
    if (!monthMap[m]) monthMap[m] = {income:0, expense:0};
    monthMap[m][t.type] += Number(t.amount);
  });
  const months = Object.keys(monthMap).sort();
  if (monthChart) monthChart.destroy();
  monthChart = new Chart(document.getElementById('monthChart'), {
    type: 'bar',
    data: {
      labels: months,
      datasets: [
        { label: 'Income', data: months.map(m=>monthMap[m].income), backgroundColor:'#1fab5e' },
        { label: 'Expense', data: months.map(m=>monthMap[m].expense), backgroundColor:'#e5484d' }
      ]
    },
    options: { plugins:{legend:{position:'bottom',labels:{boxWidth:12,font:{size:11}}}}, scales:{y:{beginAtZero:true}} }
  });
}

function renderAll(){
  renderTable();
  renderSummary();
  renderMonthSummary();
  renderCharts();
}

document.getElementById('txForm').addEventListener('submit', e => {
  e.preventDefault();
  const desc = document.getElementById('desc').value.trim();
  const amount = parseFloat(document.getElementById('amount').value);
  const type = document.getElementById('type').value;
  const category = document.getElementById('category').value;
  const date = document.getElementById('date').value;
```

```
  if (!desc || !amount || !date) return;

  if (editingId){
    const tx = transactions.find(t=>t.id===editingId);
    Object.assign(tx, {description:desc, amount, type, category, date});
    editingId = null;
    e.target.querySelector('button[type=submit]').textContent = 'Add';
  } else {
    transactions.push({id: uid(), description:desc, amount, type, category, date});
  }
  save();
  e.target.reset();
  document.getElementById('date').valueAsDate = new Date();
  renderAll();
});

window.editTx = function(id){
  const t = transactions.find(t=>t.id===id);
  if (!t) return;
  document.getElementById('desc').value = t.description;
  document.getElementById('amount').value = t.amount;
  document.getElementById('type').value = t.type;
  document.getElementById('category').value = t.category;
  document.getElementById('date').value = t.date;
  editingId = id;
  document.querySelector('#txForm button[type=submit]').textContent = 'Update';
  window.scrollTo({top:0, behavior:'smooth'});
};

window.deleteTx = function(id){
  transactions = transactions.filter(t=>t.id!==id);
  save();
  renderAll();
};

document.getElementById('catForm').addEventListener('submit', e => {
  e.preventDefault();
  const val = document.getElementById('newCategory').value.trim();
  if (val && !categories.includes(val)){
    categories.push(val);
    save();
    populateCategorySelects();
    renderCategoryList();
  }
  e.target.reset();
});

['searchBox','filterType','filterCategory','filterFrom','filterTo'].forEach(id => {
  document.getElementById(id).addEventListener('input', renderTable);
  document.getElementById(id).addEventListener('change', renderTable);
});

document.getElementById('clearFilters').addEventListener('click', () => {
  document.getElementById('searchBox').value = '';
  document.getElementById('filterType').value = '';
  document.getElementById('filterCategory').value = '';
  document.getElementById('filterFrom').value = '';
  document.getElementById('filterTo').value = '';
  renderTable();
});

// init
populateCategorySelects();
renderCategoryList();
document.getElementById('date').valueAsDate = new Date();
renderAll();
```

4.3 Project Execution

How to run the application

- Download or clone the project folder.
- Make sure the folder contains **index.html** (and **chart.umd.js** for the charts).
- Double-click **index.html** to open it in any modern browser. No installation or internet connection is required.

Working process (user input to output)

- The user opens the application and sees the dashboard with zero totals and an empty transaction list.
- The user fills the Add Transaction form (description, amount, type, category, date) and clicks Add.
- The new transaction appears instantly in the table, and the totals, monthly summary and charts update.
- The user can search by keyword, or filter by type, category or date range to narrow the list.
- The user can click Edit to modify a transaction, or Delete to remove it; all figures recalculate instantly.
- The user can add new categories from the Manage Categories panel for use in future transactions.

5. Output Screenshots

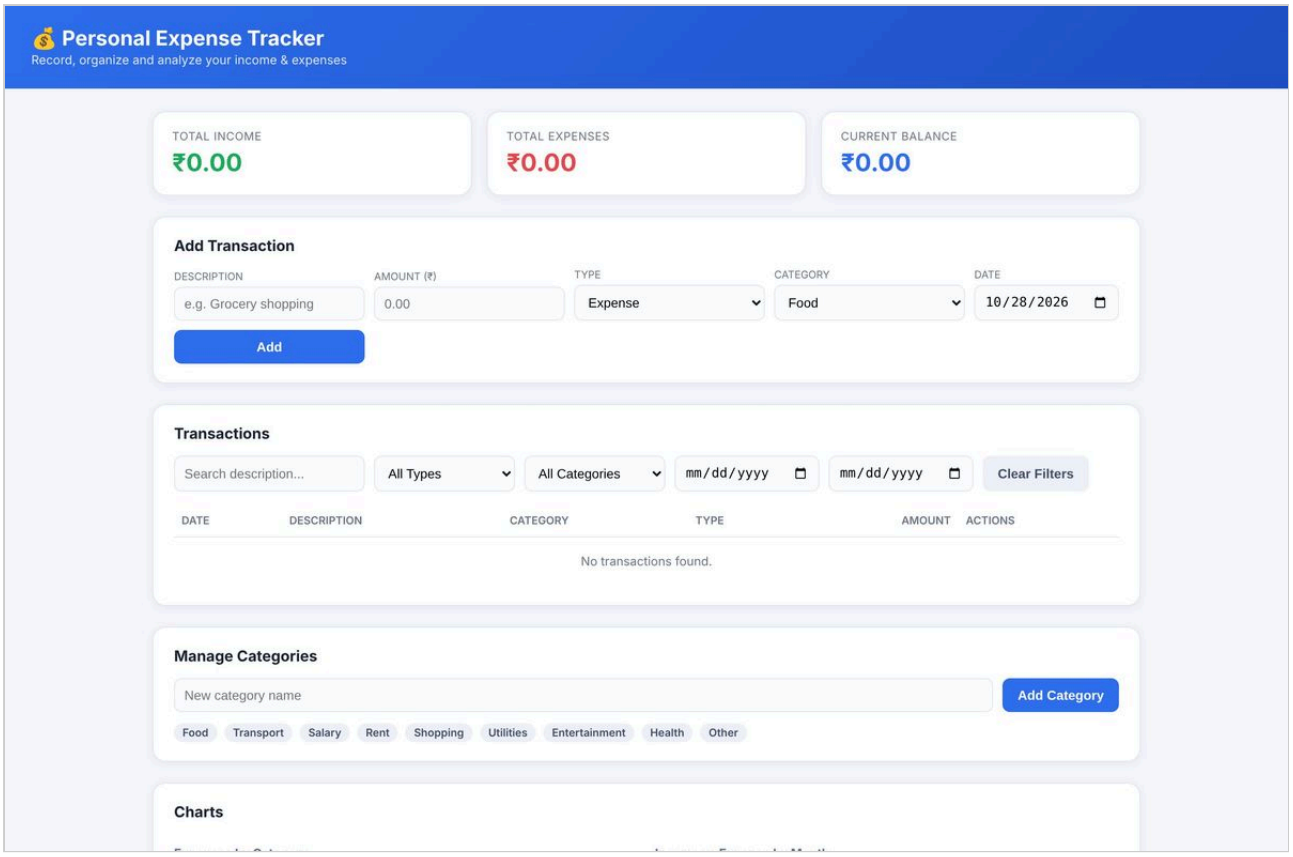


Figure 3: Dashboard on first load (Total Income, Expenses, Balance)

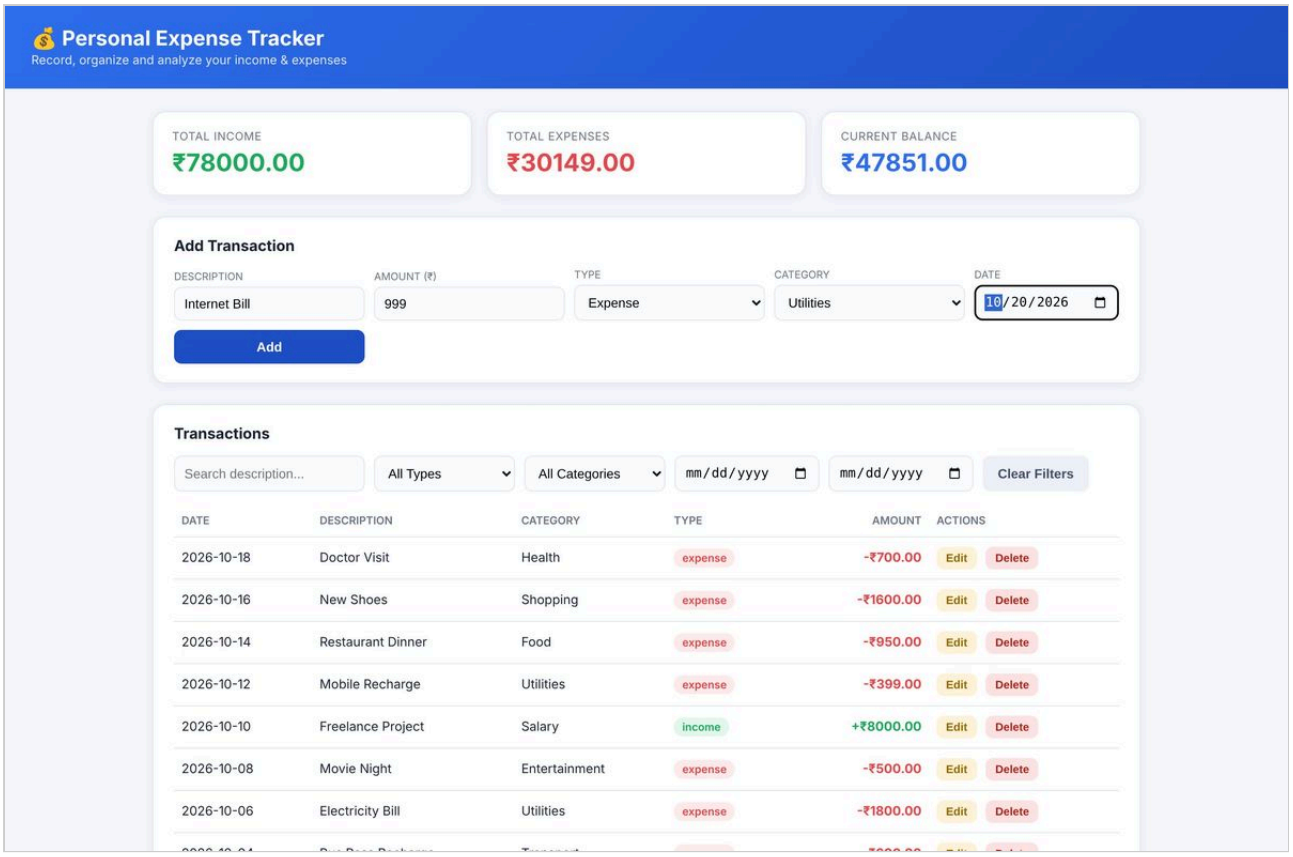


Figure 4: Add Transaction form filled in before submission

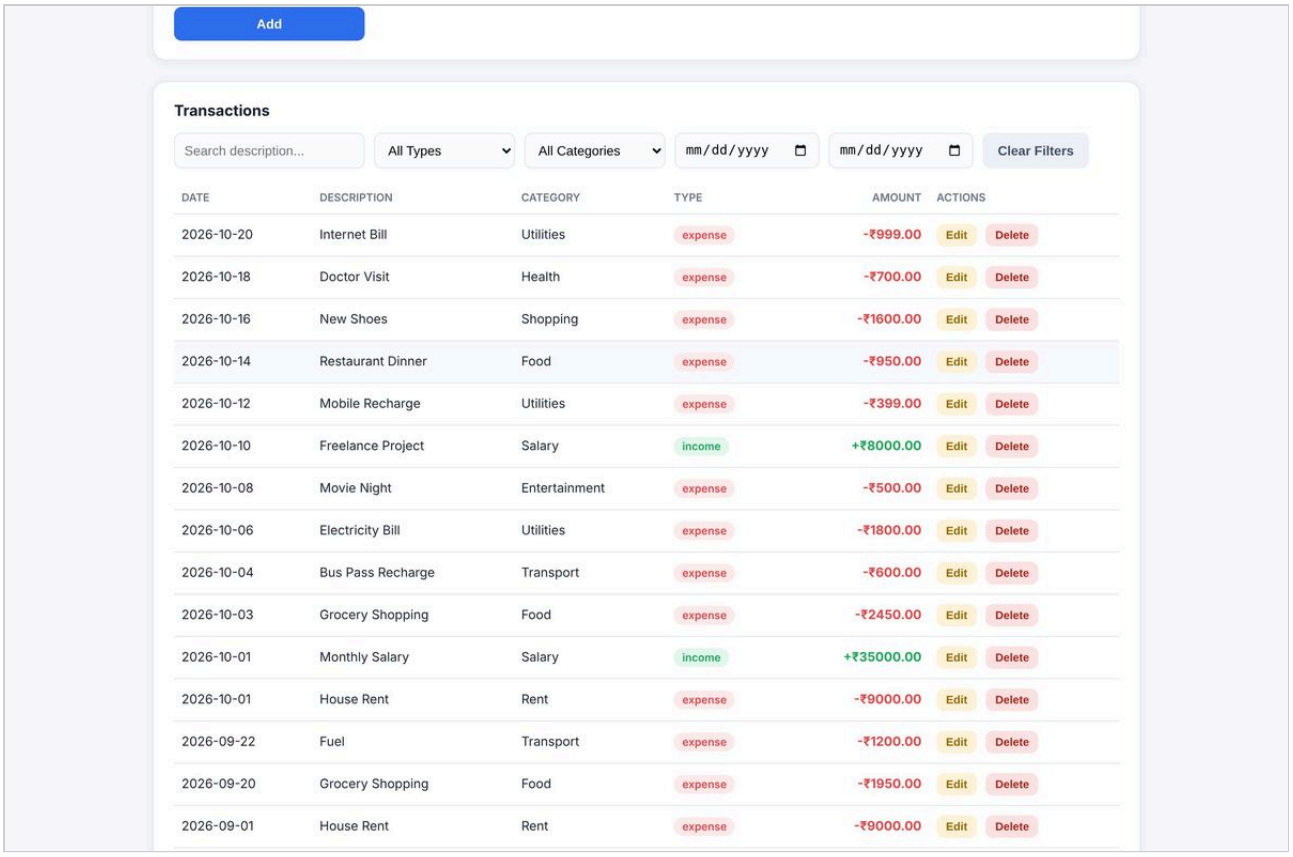


Figure 5: Transaction list showing recorded income and expense entries

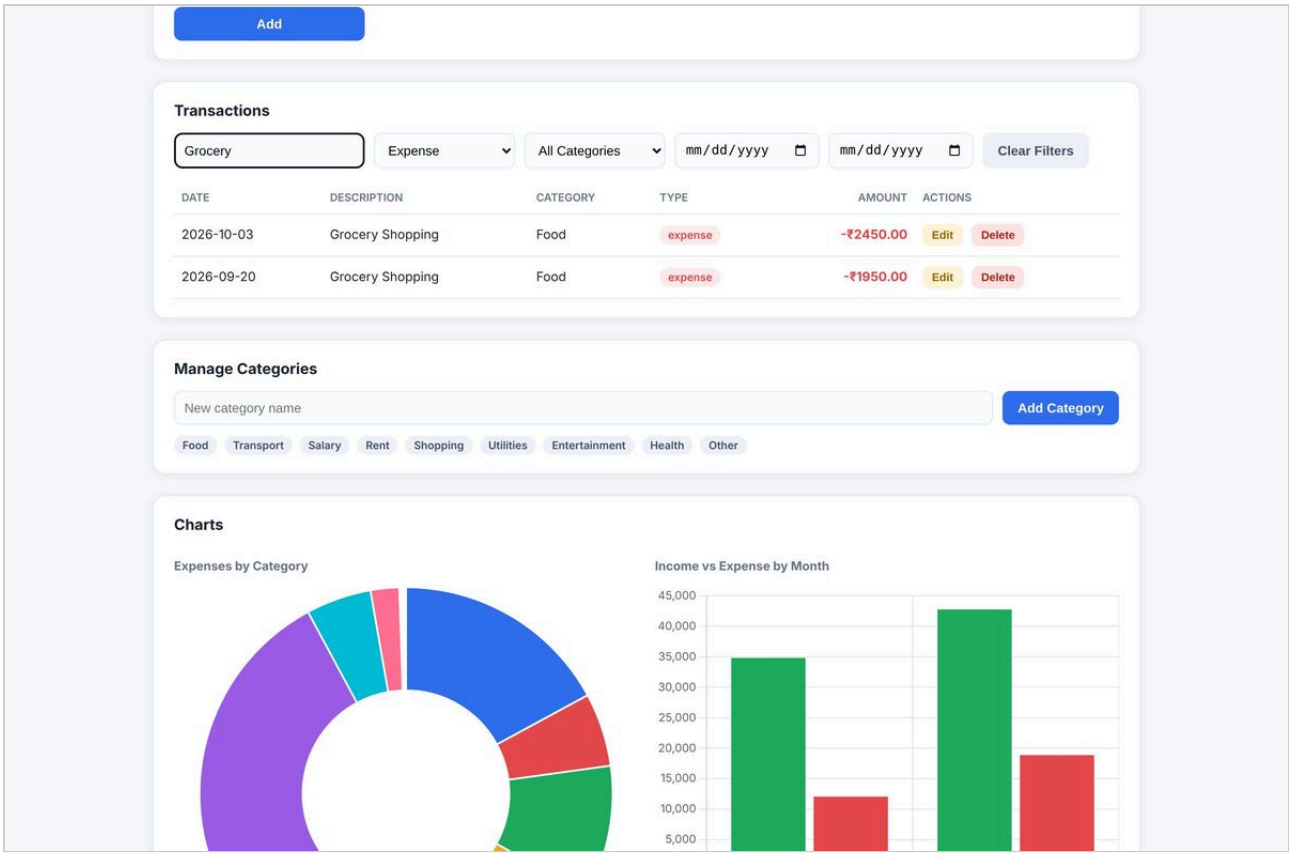


Figure 6: Search and filter in action (keyword: "Grocery", type: Expense)

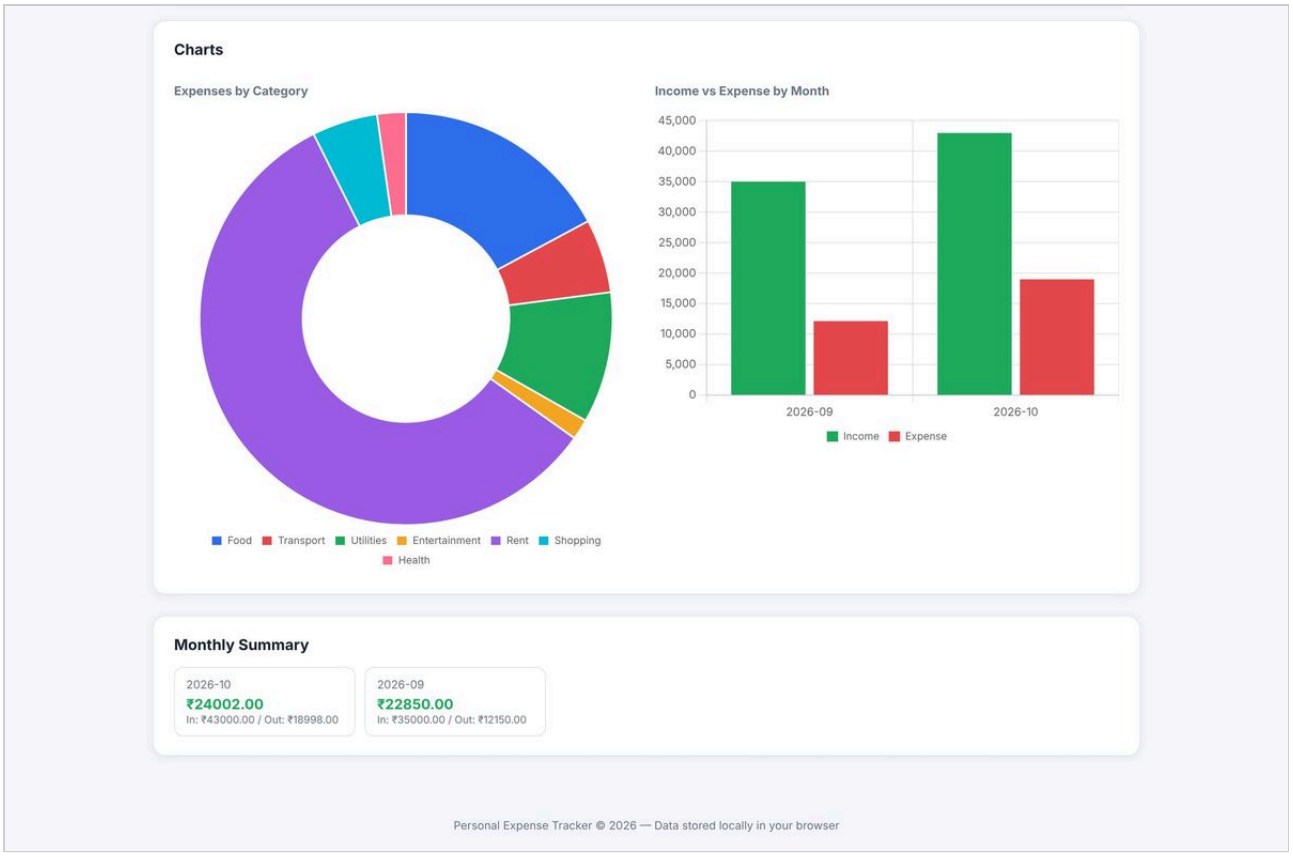


Figure 7: Expense-by-category chart and Income vs Expense bar chart

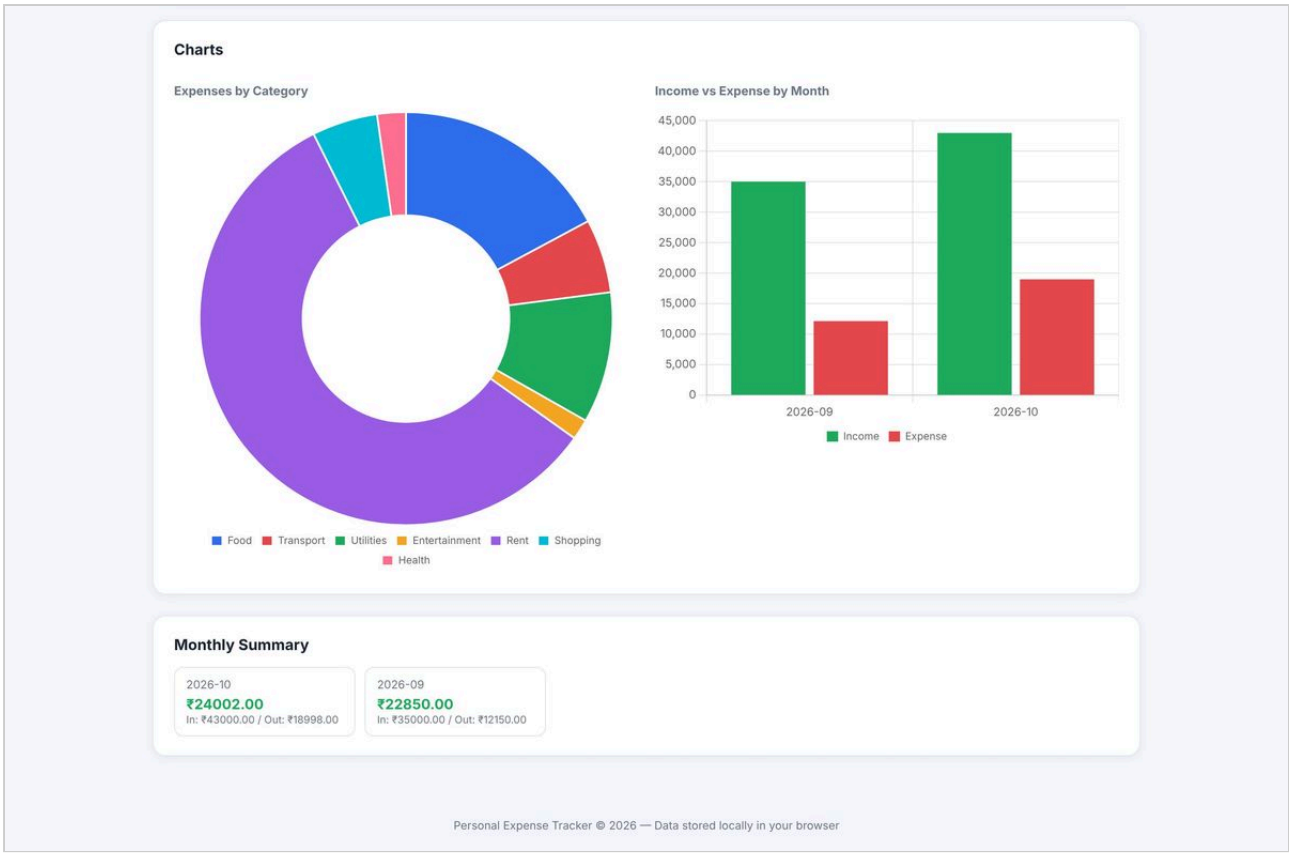


Figure 8: Monthly financial summary cards

6. Technical Explanation

- **HTML5:** Semantic structure (header, form, table, sections) organises the content and improves accessibility.
- **CSS3:** CSS variables define the colour theme; Flexbox and CSS Grid build the responsive cards, form and charts layout.
- **Data structures:** Transactions are stored as an array of objects (id, description, amount, type, category, date); categories as an array of strings.
- **DOM manipulation:** The transaction table, summary cards and category list are rebuilt from the current data using template strings and innerHTML.
- **Event handling:** Form submit events add/update transactions; input/change events on the filter fields re-render the table live.
- **Validation:** The form requires a description, a positive amount and a date before a transaction can be saved.
- **Data persistence:** JSON.stringify/parse with localStorage.setItem/getItem keeps all data available after the page is closed and reopened.
- **Data visualisation:** Chart.js renders a doughnut chart of expenses by category and a bar chart of income vs. expense by month, both updated on every change.

7. Conclusion

The Personal Expense Tracker was successfully developed using HTML, CSS and JavaScript. It lets a user record income and expenses, manage categories, search and filter transactions, and view dynamically calculated totals, monthly summaries and charts. Through this project, the core concepts of front-end web development — DOM manipulation, event handling, form validation, client-side data persistence and chart-based data visualisation — were combined into one cohesive, practical application that works entirely without a backend server.

8. Future Enhancements

- Add a back-end (Node.js / Express with a database) to store data permanently and across devices.
- User registration and login with secure password hashing and session handling.
- Export monthly/yearly reports to PDF or Excel.
- Set monthly budget goals per category with alerts when a limit is approached or exceeded.
- Recurring transaction support (e.g. automatic monthly rent or salary entries).
- Improved security (input sanitisation) and a mobile app or Progressive Web App version.